

A Library Implementation of POSIX Threads under UNIX

Frank Mueller · Florida State University · Winter USENIX 1993

paper · 5 min

simulación · 2 min

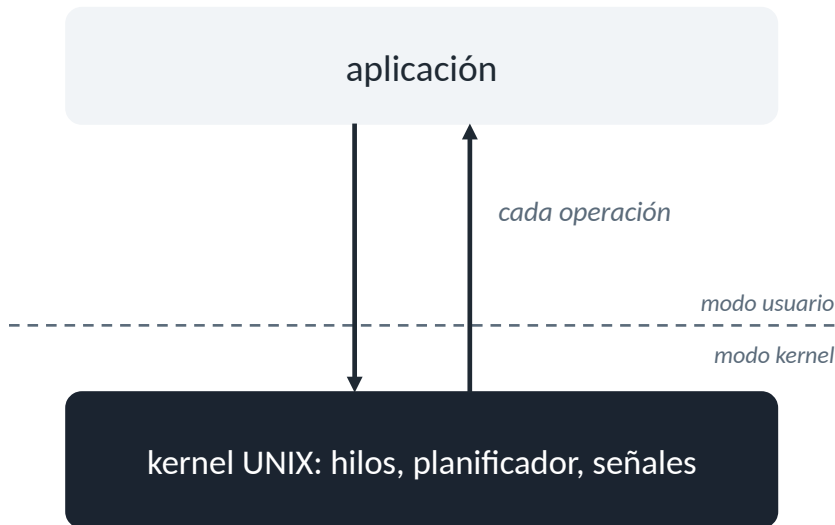
Selfie · 2 min

Gabriel Espinoza · Sistemas Operativos CS3015, 2026-2 · Proyecto 1

github.com/Gabrielee32159/mini-pthreads

¿Dónde viven los hilos?

Implementación en el kernel



18 μ s

entrar y salir del kernel UNIX

Implementación en librería (el paper)



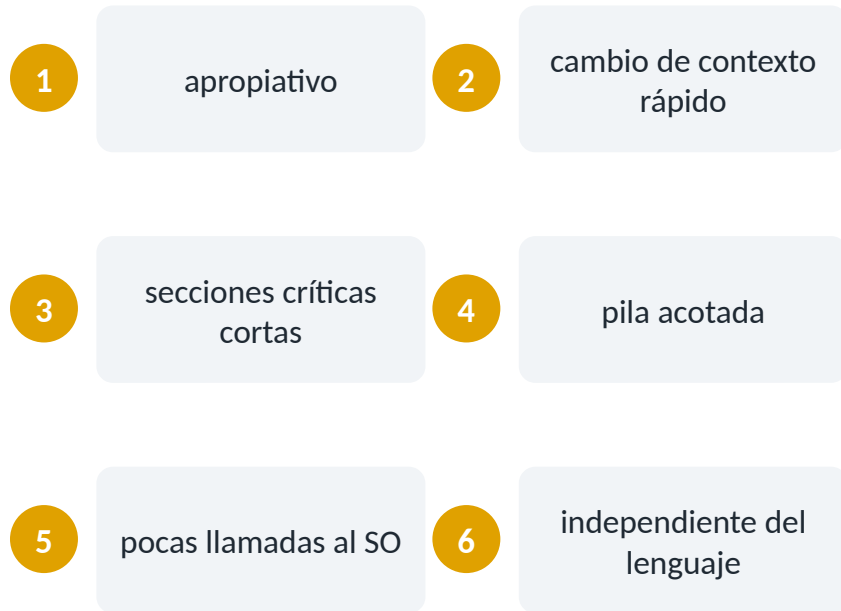
0.4 μ s

entrar y salir del kernel de la librería

La propuesta: todo en modo usuario

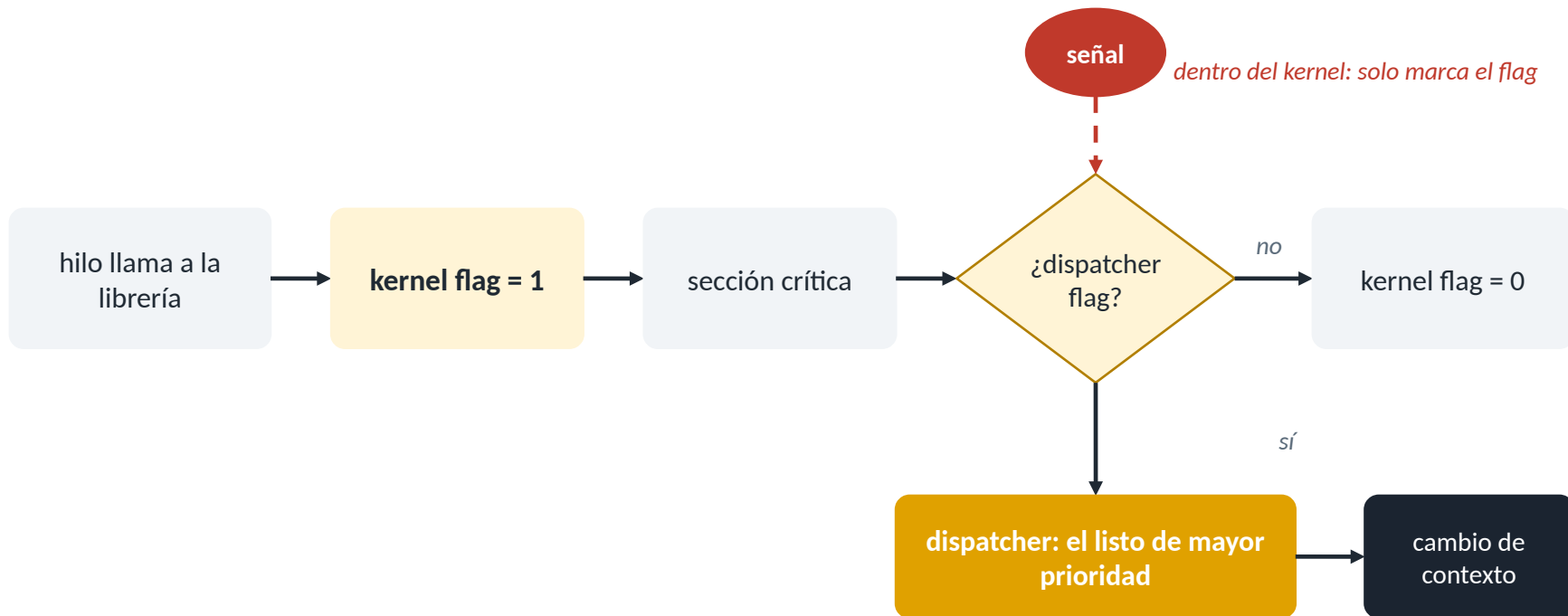


~400 líneas de ensamblador SPARC; el resto, C



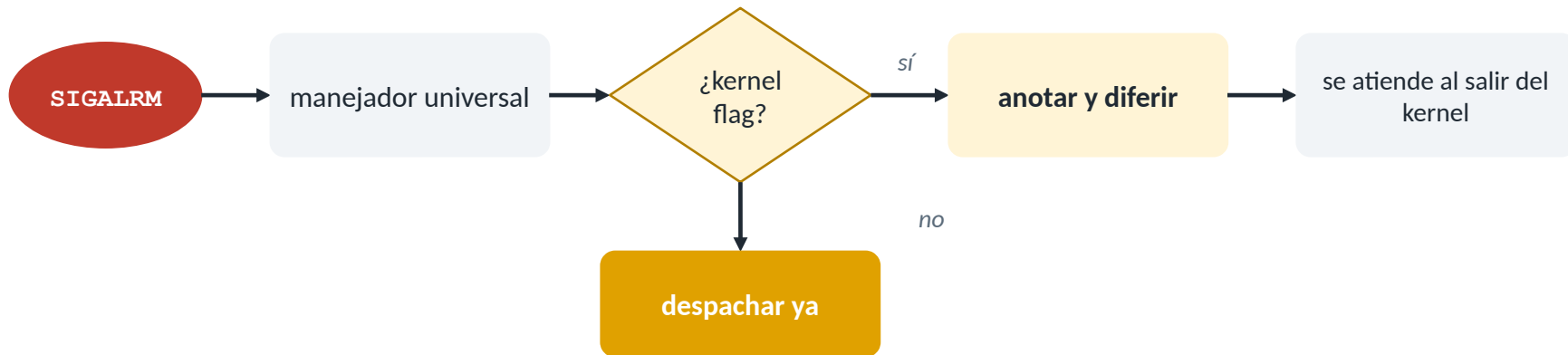
seis objetivos de diseño

El kernel de la librería: un monitor con dos flags

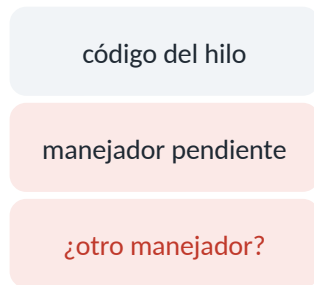


solo un hilo a la vez dentro; sin locking fino porque es uniprosesor

Señales sin entrar al kernel de UNIX

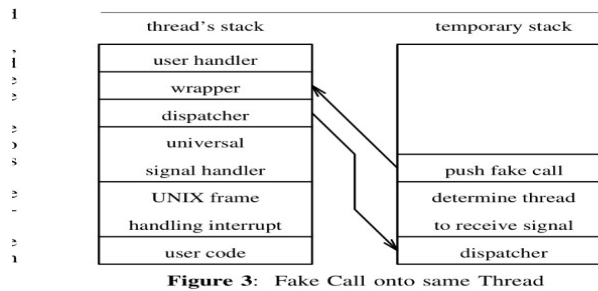


Pila acotada

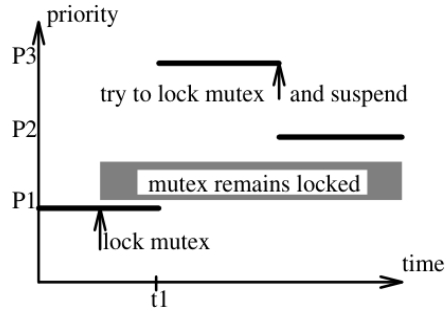


antes de volver a un hilo interrumpido se bloquean las señales; se rehabilitan al retornar del manejador

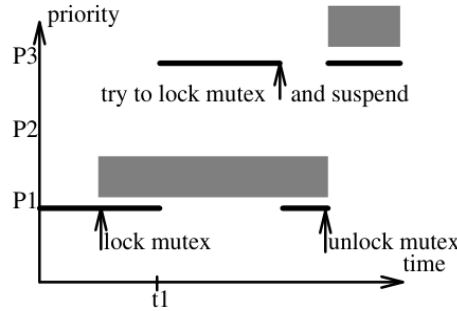
Fake call: el handler de usuario corre a la prioridad del hilo



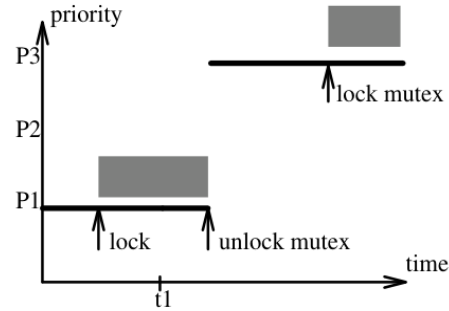
Inversión de prioridades: dos protocolos



(a) no protocol



(b) inheritance protocol



(c) ceiling protocol

Figure 5: Dealing with Priority Inversion

sin protocolo

P2 corre; P3 espera sin límite

herencia

P1 hereda la prioridad de P3

techo (SRP)

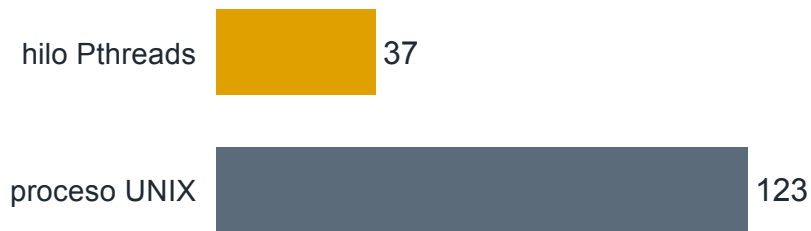
P1 sube al techo al tomar el mutex

Resultados: rinde como el kernel, o mejor

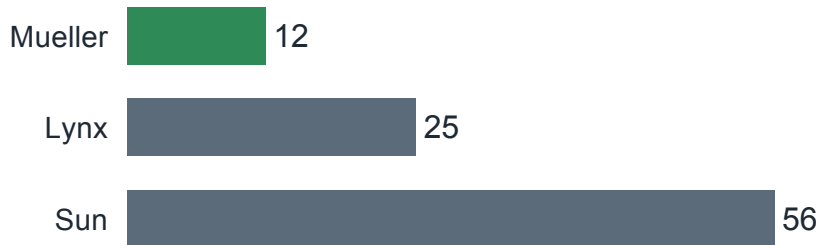
Entrar y salir del kernel (μ s)



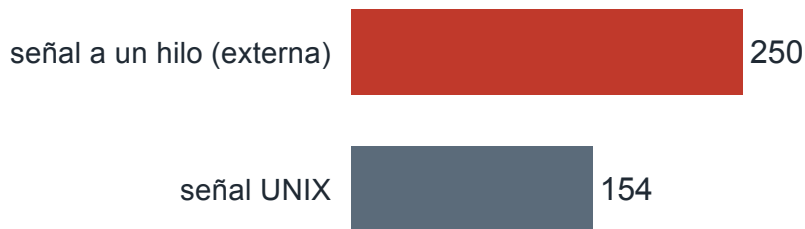
Cambio de contexto (μ s)



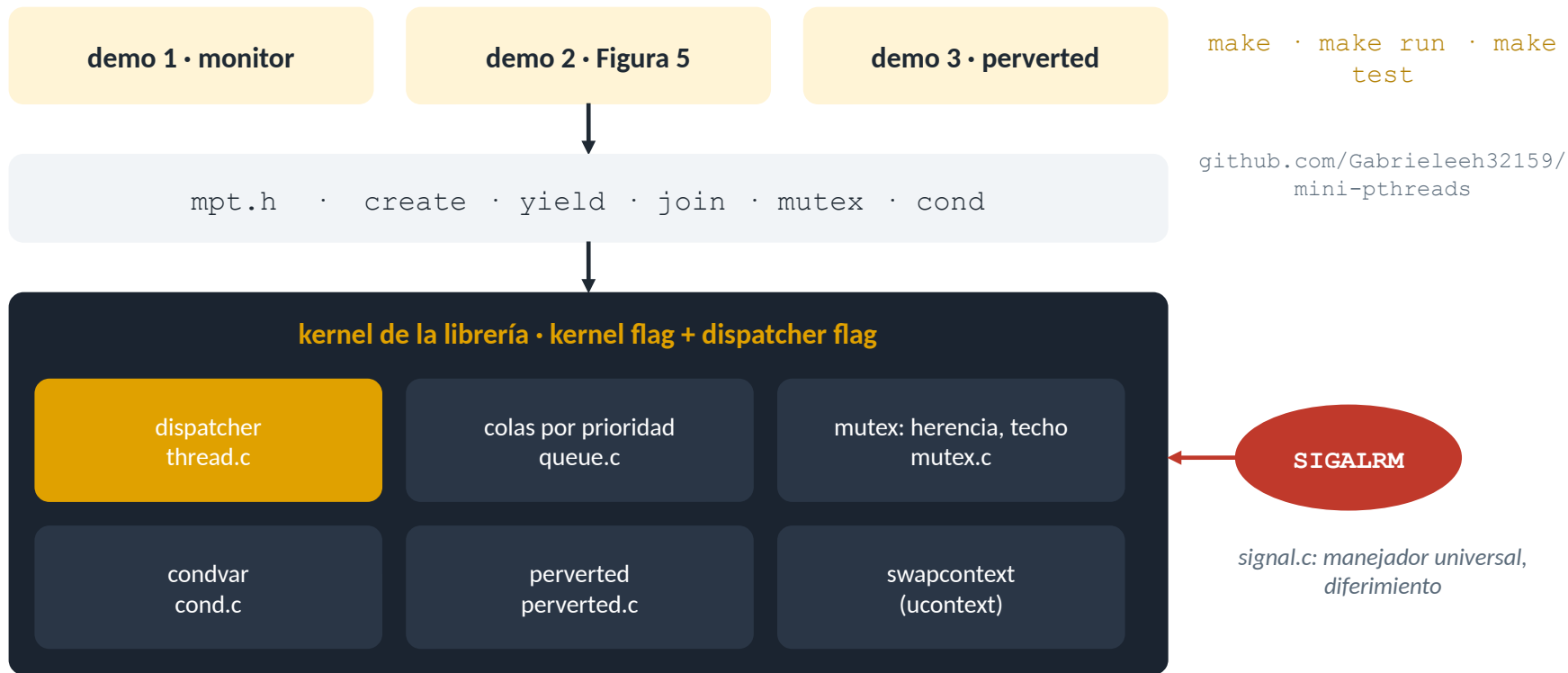
Crear un hilo (μ s)



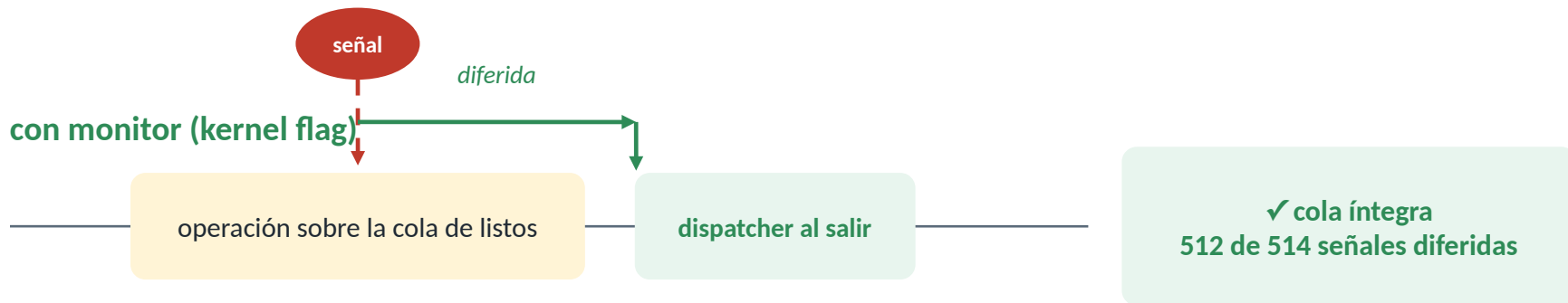
Señales: el punto débil (μ s)



Simulación: mini-pthreads



Demo 1: la señal cae dentro del kernel

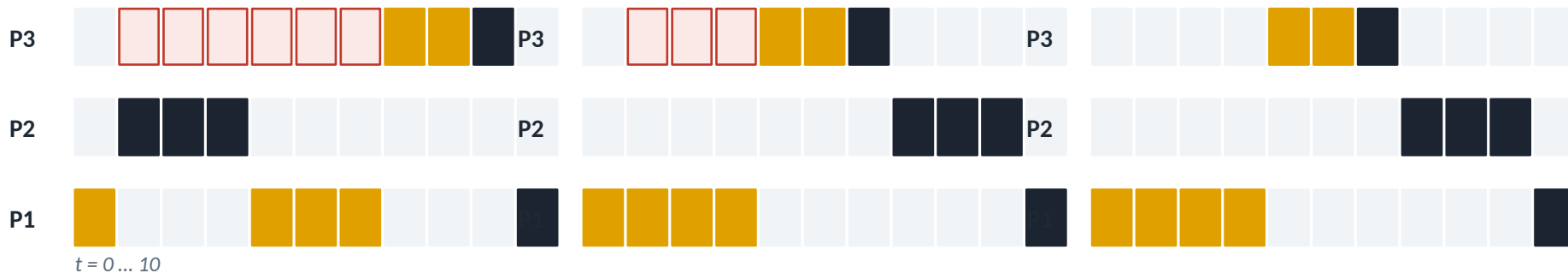


Demo 2: la Figura 5, tick a tick

sin protocolo

herencia

techo (SRP)



P3 espera 6 ticks · 10 cambios

P3 espera 3 · 9 cambios

P3 espera 3, nunca se bloquea · 7 cambios

 ejecuta

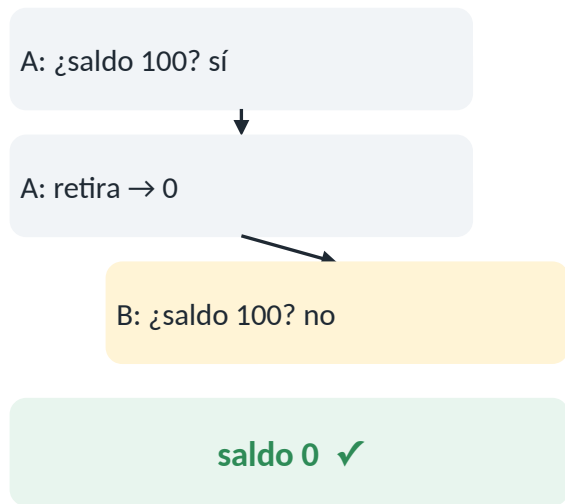
 ejecuta con el mutex

 bloqueado en el mutex

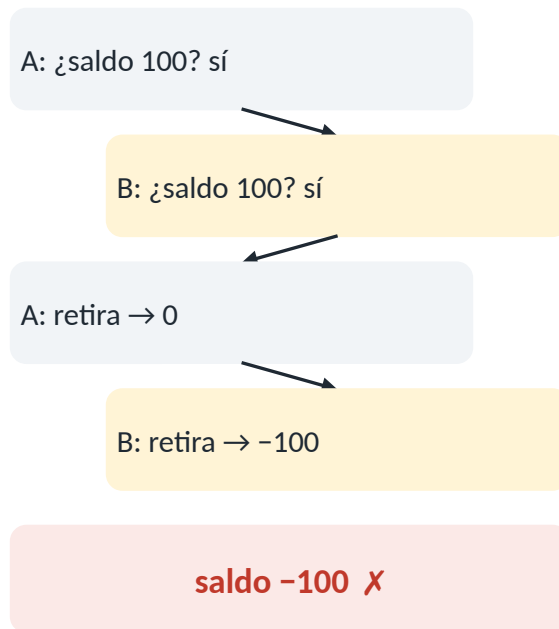
P1 baja · P2 media · P3 alta · en $t=1$ P2 y P3 pasan a listos · tiempo virtual, salida determinista

Demo 3: perverted scheduling expone la carrera

FIFO



mutex switch



la carrera existe siempre;
FIFO solo la esconde

conmutar en cada lock
la hace visible

random switch: la misma semilla,
el mismo entrelazado

check-then-act: verificar bajo el mutex, soltar, volver a tomar para retirar

Selfie: no como librería, sí dentro del emulador

✗ librería pura en C*

sin cambio de pila: no hay asm, setjmp ni registros

sin señales: no hay SIGALRM ni fake calls

sin atómicas en RISC-U: no hay test-and-set

sin struct ni punteros a función

✓ extensión de mipster / hypster

programa C*

ecall: create · yield · join · lock

mipster / hypster

contexto + prioridad, misma tabla de páginas

colas de listas por prioridad · TIMESLICE

mutex con herencia / techo

flag de perverted scheduling

Selfie: qué se conserva y qué se pierde

se conserva

inversión de prioridades y sus dos protocolos

perverted scheduling con semilla

planificación apropiativa por TIMESLICE

se pierde

la tesis central: cada operación es un ecall

fake calls y cancelación asíncrona

más de un procesador

el emulador es el monitor monolítico: nunca se interrumpe a mitad de sus colas

En una frase

Pthreads cabe entero en modo usuario si la librería trae su propio kernel: un monitor con dos flags, un manejador que difiere señales y un cambio de contexto barato.

0.4 μ s vs 18 μ s

paper

3 demos

mini-pthreads

en el emulador

Selfie